

3.3 RQ3: Agentic Task Suitability

On average, Survey participants rated the agents' suitability for their tasks as 4.69/6 (1="Extremely unsuitable", 6="Extremely suitable", see Fig. 2). Both Observations and Survey participants identified successful and unsuccessful use cases for agents.

Suitable: Improving Productivity. 9 Observation participants mentioned how coding with agents has accelerated coding tasks, taking note of how much time they save, the increase in speed, and how much they can output in a short period of time (3x Observations). 2 Observational participants also mentioned how agents provide them with personalized solutions to their specific user case, with P2 explaining that *"...it seems like the equivalent of a Stack Overflow answer tailored to my specific scenario."* Agents are also suitable for small tasks that are easy to test (P8). The agent's performance on simple tasks with narrow scopes is notable to experienced developers, with some users praising this part of their workflow (21x Survey). Agents are also suitable for version control review. Some observational participants use agents for reviewing pull requests on GitHub (P5, P6, P9) and P10 mentioned how he uses them to write *"better quality Git commit messages"*, detailing that *"...if I ever do need to go back through time to search through things, my commit messages are great because AI... does commit messages."* Similarly, Agents are suitable for common tasks like documentation and scaffolding. Five Observations participants (P5, P6, P7, P8, P10) echoed this sentiment, with P8 capturing this idea by stating he integrated agents into his workflow to *"mainly save... from the really boring, tedious boilerplate code... That and unit tests is mostly the stuff I don't like to do myself for the most part."* Furthermore, 50 survey respondents agreed with this idea of handing off mundane tasks they don't feel like doing themselves to the agent.

Suitable: Filling in Knowledge Gaps. Agents enable creativity and exploration for the user, with 3 Observational participants and 4 survey respondents explaining how their agentic coding workflows help them think of more ideas, pointing out unfamiliar concepts, and allowing them to stay in the design phase longer. P13 mentioned that he *"explicitly asked [the agent] to challenge [him] and [...] double check what [he was] proposing."* Additionally, agents are suitable for articulating unfamiliar code or concepts. For example, P12 explains that when he needs to architect systems, agents are a *"lifesaver"* because he can ask them to draw the context of the system and how it fits in with a particular feature he has in mind, allowing him to get a visual representation of what the system looks like, expediting his design process. By extension, agents help experienced developers build software in unfamiliar domains (4x Survey). 4 Observational participants detailed that with their workflows, they are able to build software they are not well versed in, speeding up the process of building prototypes which would have taken them a long time otherwise. This bridging of unfamiliar domains can even impact how they learn: *"I'm very comfortable reading modern React code at this point, but only after the last few months of [...] collaborating with AI and learning from it."* (P3)

Mixed: Improving Code Quality. Agent suitability for improving code quality had mixed opinions with both sets of our participants. For those with successful cases of obtaining high-quality code quickly, 4 Observational participants detailed that the agentic coding solution was fast and accurate. P3 outlined that a migration to React was completed in three days,

specifically explaining that, *"It would have taken probably two weeks of my own time... And at the end of three days, I don't think we've caught a single bug or... design mistake ever since then."* 9 survey respondents furthered these claims, explaining that agents were good at generating code based on an example they fed into a prompt, only having to fix small mistakes after an initial pass through.

For those with more negative experiences, they first mentioned that agents are unsuitable for implementing proper engineering practices. 6 Observational participants described this failure using quality attributes in software engineering, explaining that the code written by the agent can sometimes be bad for testing, be overly verbose, or not follow standard data privacy and security measures. Moreover, the agents can simply struggle with correctness, not allowing the user to build on existing functionality and having to debug the code that was generated. Indeed, S96 explains that the agent *"did not do bad, just behaved as a human, so we both had to work off each other's work,"* The context in which this code generation is taking place also matters, as 24 survey respondents detailed that agents do not perform well at making high-level architectural decisions or making real-world choices informed by business logic, with S68 stating that they *"never trust LLMs for systematic issues."* Notably, experienced developers were aware of this limitation and made sure to plan and verify their goals when interacting with the agent in order to avoid flawed architectures. As S73 put it: *"I prefer to handle the high-level, strategic responsibilities without assistance from a code agent. This includes tasks like project planning, making architectural decisions, and choosing a technology stack."*

Finally, participants shared differing opinions about agents' suitability for developing quality user interfaces. Some participants found agents extremely helpful (2x Observations, 5x Survey) when they had little knowledge about UIs: *"I don't have much experience in UI-type [of] things [...] so having [agents] generate something visual to a program URL, that's really nice [...] that's something that would take me a really long time to figure out myself."* In contrast, some other participants found the chat-based interaction for agentic UI improvements insufficient (2x Observations, 9x Survey). P1 shared his failure of using agents to implement the desired visual aesthetics of an animated web banner, struggling with accurately specifying his intent: *"I [tried], I don't know, 30 times [...] And no matter how I prompted it, [it] just wouldn't achieve what I was looking for."*

Mixed: Implementing Complex Tasks. Participant had mixed opinions about agent use on complex tasks. For some participants, agents are suitable for untangling the complexity, decomposing large tasks into smaller implementation steps (S1, S25). We also had 5 Observational participants say that agents were able to bring them closer to their software outcome, again, bringing up how getting a prototype up and running is much quicker now. P10 explain this more directly by stating, *"...I've been coding for a really long time and ...even just with my role now is more to get things done quick... And I think that like I've had enough experience building things in the right way."*

Mixed: Navigating Codebase Context. For some users (2x Observations), the agent was suitable for processing context, specifically for reminding the user of forgotten parts of the

codebase. On this point, P4 explained that when the agent would scan the files and make changes to behavior, it found other parts of the code it needed to change that he did not even remember, and some survey respondents detail that the overall context performance of their chosen agent did well, with S96 saying that it was *"Quickly onboarding itself with the necessary setup and environment info."* For those with less than positive experiences, their concerns were mostly rooted in agents not being able to navigate through a large context space. This came in several forms. First, agent hallucinations remained an issue and some users brought up that agents could sometimes do more than asked, with 8 Observations participants explaining that it can create unnecessary files, rewrite entire sections of code, or install packages that are not needed for the task at hand. Put bluntly, P7 said, *"I do not quite enjoy the fact that it will just like start changing and changing stuff without me actually understanding what's going on."* On a larger scale, 20 survey respondents mentioned general issues with agent context management, explaining that agents can sometimes have difficulty ensuring that complex edge cases, feature integration, and the correct details are properly fleshed out for them. There is also a difficulty on maintaining context between and within chats, as P5 states that *"it's so hard to leave that context cache because [...] I've spent so many iterations with already of just trying to get to [...] do that first part. And now I'm like, OK, now we're going to build on top of it. And then it starts getting confused."*

Other Unsuitable Tasks. For some users, understanding agent behavior remains a pain point. 3 Observational participants mentioned that the agent would get stuck or "hang" trying to produce a solution, which can waste both time and money for the user. P12 elucidates that *"if you're using Cursor and it starts that spiraling [...] now they have a limited amount of requests which you're [...] burning up and I am going to waste a lot of my budget right on on failed calls, which is [...] bad for us as users."* This issue is made worse for those that use background agents or orchestration platforms for parallel agent use, like P9 did. He mentioned that due to the nature of his orchestration tool Terragon, they don't do a good job at giving a high level view of the work currently being done and *"...it's very hard to... see what's actually going on."* 6 survey respondents detailed how agents do not perform well with newer or less common tools, acknowledging that training data for these tools are not that present on the web. S7 explains this more specifically by saying, *"They are surprisingly bad at operating Claude itself (changing it's config, configuring new MCP servers, that sort of thing). This makes sense, because everything about it is relatively new and probably from after they were trained. But it's a bit ironic."* Finally, there is a lack of trust with agents running low-level commands in your system, with P6 specifically mentioning that *"I don't really trust it enough for that yet."* Likewise, this is echoed by 7 survey respondents, with S79 not trusting the agent to run Git commands, detailing he mostly does it himself.

Takeaway 3: Experienced developers find agents suitable for improving development speed (e.g., automating small and common tasks) and filling in human knowledge gaps (e.g., for exploring ideas and unfamiliar concepts), but share mixed experiences with agents in tasks relevant to code quality, task complexity, large codebases, less common tools, system programming, and debugging of the agentic behavior.